



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Online Food Order Management and Analysis System

CSA0801 – Python Programming

[S R Anusika] ([1920260008])

Pseudo code:

```
// SYSTEM ARCHITECTURE & UNIFIED PSEUDOCODE
// Project: Online Food Order Management and Analysis System
// Language: Python 3 (Console / Menu-driven)
// Persistence: JSON file (food_order_data.json)

// MODULE 1: GLOBAL DATA STRUCTURES
DICTIONARY customers      // {customer_id: {name, phone, email, address}}
DICTIONARY restaurants    // {restaurant_id: {name, cuisine, rating, menu}}
DICTIONARY orders         // {order_id: {customer_id, restaurant_id, items, status, ...}}
TUPLE ORDER_STATUSES = ("Placed", "Preparing", "Out for Delivery", "Delivered", "Cancelled")
SET FOOD_CATEGORIES = {"Starter", "Main Course", "Dessert", "Beverage"}

// MODULE 2: CUSTOM EXCEPTIONS
CLASS InvalidInputError EXTENDS Exception
CLASS RecordNotFoundError EXTENDS Exception

// MODULE 3: INPUT VALIDATION HELPERS
FUNCTION read_non_empty(prompt):
    value = INPUT(prompt).strip()
    IF value IS empty: RAISE InvalidInputError
    RETURN value

FUNCTION read_positive_int / read_positive_float(prompt):
    TRY: value = CONVERT(INPUT(prompt))
    CATCH ValueError: RAISE InvalidInputError("not a valid number")
    IF value <= 0: RAISE InvalidInputError("must be positive")
    RETURN value

// MODULE 4: CUSTOMER MANAGEMENT
FUNCTION register_customer():
    TRY:
        name, phone, email, address = read_non_empty(...) each
    CATCH InvalidInputError: PRINT error AND RETURN
    customers[next_customer_id] = {name, phone, email, address}
    INCREMENT next_customer_id
    PRINT "Customer registered, ID = ..."

FUNCTION view_customers():
    IF customers is empty: PRINT "No customers" ; RETURN
    FOR EACH (id, c) IN customers: PRINT formatted row
```

// MODULE 5: RESTAURANT & MENU MANAGEMENT

```
FUNCTION add_restaurant():
    TRY: name, cuisine, rating = read inputs; VALIDATE rating <= 5
    CATCH InvalidInputError: PRINT error; RETURN
    restaurants[next_restaurant_id] = {name, cuisine, rating, menu: {}}
    INCREMENT next_restaurant_id

FUNCTION add_food_item():
    TRY:
        rid = read_positive_int("restaurant id")
        IF rid NOT IN restaurants: RAISE RecordNotFoundError
        name, price = read inputs
        category = read_non_empty(...)
        IF category NOT IN FOOD_CATEGORIES: RAISE InvalidInputError
    CATCH (InvalidInputError, RecordNotFoundError): PRINT error; RETURN
    restaurants[rid].menu[next_item_id] = (name, price, category)
    INCREMENT next_item_id

FUNCTION view_restaurants():
    FOR EACH (rid, r) IN restaurants:
        PRINT restaurant header (name, cuisine, rating)
        FOR EACH (item_id, (name, price, category)) IN r.menu:
            PRINT formatted menu row
```

// MODULE 6: BILLING ENGINE

```
FUNCTION calculate_bill(item_list, restaurant):
    subtotal = SUM(price[item] * qty FOR item, qty IN item_list)
    IF subtotal > 500: discount = subtotal * 0.10
    ELSE IF subtotal > 250: discount = subtotal * 0.05
    ELSE: discount = 0
    net = subtotal - discount
    delivery_charge = 0 IF net > 300 ELSE 40
    total = ROUND(net + delivery_charge, 2)
    RETURN subtotal, discount, delivery_charge, total
```

// MODULE 7: ORDER MANAGEMENT

```
FUNCTION place_order():
    TRY:
        cid = read_positive_int("customer id"); VALIDATE EXISTS
        rid = read_positive_int("restaurant id"); VALIDATE EXISTS
        item_list = []
        WHILE TRUE:
            item_id = INPUT("item id, 0 to finish")
            IF item_id == 0: BREAK
            IF item_id NOT IN restaurant.menu: PRINT error; CONTINUE
            qty = read_positive_int("quantity")
            item_list.APPEND((item_id, qty))
            IF item_list is empty: RAISE InvalidInputError
    CATCH (InvalidInputError, RecordNotFoundError): PRINT error; RETURN
    subtotal, discount, delivery_charge, total = calculate_bill(item_list, restaurant)
    orders[next_order_id] = {customer_id: cid, restaurant_id: rid, items: item_list,
        status: "Placed", order_time: NOW(),
        subtotal, discount, delivery_charge, total}
    INCREMENT next_order_id
```

```

PRINT "Order placed, ID = ..., Total = ..."

FUNCTION update_order_status():
    TRY:
        oid = read_positive_int("order id"); VALIDATE EXISTS
        new_status = read_non_empty("new status")
        IF new_status NOT IN ORDER_STATUSES (case-insensitive): RAISE InvalidInputError
    CATCH error: PRINT error; RETURN
    orders[oid].status = new_status
    PRINT "Order status updated"

FUNCTION view_order_details(order_id):
    VALIDATE order_id EXISTS
    o = orders[order_id]
    PRINT customer name, restaurant name, order time, status
    FOR EACH (item_id, qty) IN o.items: PRINT item name, qty, price
    PRINT subtotal, discount, delivery charge, TOTAL

// MODULE 8: CUSTOM OPERATION 1 - COMPARE TWO ORDERS
FUNCTION compare_orders():
    TRY:
        oid1 = read_positive_int("first order id")
        oid2 = read_positive_int("second order id")
        VALIDATE both EXIST
    CATCH error: PRINT error; RETURN
    t1 = orders[oid1].total ; t2 = orders[oid2].total
    IF t1 > t2: PRINT "order1 has higher bill, difference = t1 - t2"
    ELSE IF t2 > t1: PRINT "order2 has higher bill, difference = t2 - t1"
    ELSE: PRINT "both totals are equal"

// MODULE 9: CUSTOM OPERATION 2 - PERFORMANCE REPORT
FUNCTION generate_report():
    IF orders is empty: PRINT "no orders" ; RETURN
    total_orders = COUNT(orders)
    all_totals = [o.total FOR o IN orders]
    total_revenue = SUM(all_totals)
    avg_order_value = MEAN(all_totals)
    status_count = COUNT orders GROUPED BY status
    restaurant_revenue = SUM(o.total) GROUPED BY restaurant_id
    best_restaurant = restaurant_id WITH MAX(restaurant_revenue)
    PRINT report header, generated timestamp
    PRINT total customers, restaurants, orders, revenue, average order value
    PRINT top performing restaurant + revenue
    PRINT order status breakdown FOR EACH status IN ORDER_STATUSES

// MODULE 10: FILE HANDLING (PERSISTENCE)
FUNCTION save_data():
    TRY:
        data = {customers, restaurants, orders, next_ids}
        WRITE data AS JSON TO "food_order_data.json"
        PRINT "Data saved"
    CATCH OSError: PRINT "Could not save data"

FUNCTION load_data():

```

```

IF file NOT EXISTS: PRINT "starting fresh"; RETURN
TRY:
    data = READ JSON FROM "food_order_data.json"
    RESTORE customers, restaurants, orders, next_ids FROM data
    PRINT "Data loaded"
CATCH (OSError, JSONDecodeError): PRINT "Could not load data"

// MODULE 11: MAIN MENU / APPLICATION ENTRY POINT
FUNCTION main():
    load_data()
    WHILE TRUE:
        DISPLAY MENU (13 options)
        choice = INPUT("Enter choice")
        TRY:
            SWITCH choice:
                CASE 1: register_customer()
                CASE 2: add_restaurant()
                CASE 3: add_food_item()
                CASE 4: view_restaurants()
                CASE 5: place_order()
                CASE 6: update_order_status()
                CASE 7: view_order_details()
                CASE 8: compare_orders()
                CASE 9: generate_report()
                CASE 10: view_customers()
                CASE 11: save_data()
                CASE 12: load_data()
                CASE 13: save_data(); PRINT "Goodbye"; BREAK loop
            DEFAULT: PRINT "Invalid choice"
        CATCH Exception AS e:
            PRINT "Unexpected error:", e // safety net so menu never crashes

START main()

```

Implementation & Results:

food_order_system.py

```
"""
Online Food Order Management and Analysis System
CSA0801 - Python Programming
-----
A menu-driven console application that manages restaurants, food items,
customers and delivery orders for a food-delivery platform.

Core Python concepts demonstrated:
- Data structures : dict, list, tuple, set
- Functions & modules (json, os, datetime, statistics)
- Control flow      : if/elif/else, for, while
- String handling   : formatting, validation, report building
- Exception handling, input validation
- File handling (persistence to JSON)
- Custom operations: compare_orders(), generate_report()
"""

import json
import os
import statistics
from datetime import datetime

# -----
# GLOBAL DATA STRUCTURES
# -----
customers = {}      # {customer_id: {"name", "phone", "email", "address"}}
restaurants = {}    # {restaurant_id: {"name", "cuisine", "rating", "menu": {item_id: (name,
price, category)}}}
orders = {}         # {order_id: {...}}

ORDER_STATUSES = ("Placed", "Preparing", "Out for Delivery", "Delivered", "Cancelled")
FOOD_CATEGORIES = {"Starter", "Main Course", "Dessert", "Beverage"}

DATA_FILE = "food_order_data.json"

next_customer_id = 1
next_restaurant_id = 1
next_item_id = 1
next_order_id = 1

# -----
# CUSTOM EXCEPTIONS
# -----
class InvalidInputError(Exception):
    """Raised when user input fails validation."""
    pass

class RecordNotFoundError(Exception):
    """Raised when a customer/restaurant/order id does not exist."""
    pass

# -----
```

```

# VALIDATION HELPERS
# -----
def read_non_empty(prompt):
    value = input(prompt).strip()
    if not value:
        raise InvalidInputError("Input cannot be empty.")
    return value

def read_positive_float(prompt):
    try:
        value = float(input(prompt).strip())
    except ValueError:
        raise InvalidInputError("Please enter a valid number.")
    if value <= 0:
        raise InvalidInputError("Value must be greater than zero.")
    return value

def read_positive_int(prompt):
    try:
        value = int(input(prompt).strip())
    except ValueError:
        raise InvalidInputError("Please enter a valid whole number.")
    if value <= 0:
        raise InvalidInputError("Value must be a positive whole number.")
    return value

# -----
# CUSTOMER MANAGEMENT
# -----
def register_customer():
    global next_customer_id
    try:
        name = read_non_empty("Enter customer name: ")
        phone = read_non_empty("Enter phone number: ")
        email = read_non_empty("Enter email: ")
        address = read_non_empty("Enter delivery address: ")
    except InvalidInputError as e:
        print(f"[ERROR] {e}")
        return

    customer_id = next_customer_id
    customers[customer_id] = {
        "name": name, "phone": phone, "email": email, "address": address
    }
    next_customer_id += 1
    print(f"[OK] Customer registered successfully. Customer ID = {customer_id}")

def view_customers():
    if not customers:
        print("No customers registered yet.")
        return
    print("\n{<5}{:<20}{:<15}{:<25}{:<30}".format("ID", "Name", "Phone", "Email",
"Address"))
    print("-" * 95)
    for cid, c in customers.items():

```

```

        print("{:<5}{:<20}{:<15}{:<25}{:<30}".format(cid, c["name"], c["phone"], c["email"],
c["address"]))

# -----
# RESTAURANT & MENU MANAGEMENT
# -----
def add_restaurant():
    global next_restaurant_id
    try:
        name = read_non_empty("Enter restaurant name: ")
        cuisine = read_non_empty("Enter cuisine type: ")
        rating = read_positive_float("Enter rating (1.0 - 5.0): ")
        if rating > 5:
            raise InvalidInputError("Rating cannot exceed 5.0")
    except InvalidInputError as e:
        print(f"[ERROR] {e}")
        return

    rid = next_restaurant_id
    restaurants[rid] = {"name": name, "cuisine": cuisine, "rating": rating, "menu": {}}
    next_restaurant_id += 1
    print(f"[OK] Restaurant added successfully. Restaurant ID = {rid}")

def add_food_item():
    global next_item_id
    try:
        rid = read_positive_int("Enter restaurant ID: ")
        if rid not in restaurants:
            raise RecordNotFoundError(f"Restaurant ID {rid} not found.")
        item_name = read_non_empty("Enter food item name: ")
        price = read_positive_float("Enter price (Rs.): ")
        category = read_non_empty(f"Enter category {tuple(FOOD_CATEGORIES)}: ").title()
        if category not in FOOD_CATEGORIES:
            raise InvalidInputError(f"Category must be one of {FOOD_CATEGORIES}")
    except (InvalidInputError, RecordNotFoundError) as e:
        print(f"[ERROR] {e}")
        return

    item_id = next_item_id
    restaurants[rid]["menu"][item_id] = (item_name, price, category)
    next_item_id += 1
    print(f"[OK] Food item added. Item ID = {item_id}")

def view_restaurants():
    if not restaurants:
        print("No restaurants available.")
        return
    for rid, r in restaurants.items():
        print(f"\nRestaurant ID: {rid} | {r['name']} | Cuisine: {r['cuisine']} | Rating:
{r['rating']}")
        if not r["menu"]:
            print("    (no menu items yet)")
            continue
        print("    {:<5}{:<20}{:<10}{:<15}".format("ItemID", "Name", "Price", "Category"))
        for iid, (name, price, category) in r["menu"].items():

```

```

        print("    {:<5}{:<20}{:<10}{:<15}".format(iid, name, f"Rs.{price:.2f}",
category))

# -----
# ORDER MANAGEMENT
# -----
def calculate_bill(item_list, restaurant):
    """Returns (subtotal, discount, delivery_charge, total) for a list of (item_id, qty)."""
    subtotal = 0.0
    for item_id, qty in item_list:
        name, price, category = restaurant["menu"][item_id]
        subtotal += price * qty

    # Discount rule: 10% off orders above Rs.500, else 5% above Rs.250
    if subtotal > 500:
        discount = subtotal * 0.10
    elif subtotal > 250:
        discount = subtotal * 0.05
    else:
        discount = 0.0

    # Delivery charge: free above Rs.300 net, else flat Rs.40
    net_after_discount = subtotal - discount
    delivery_charge = 0.0 if net_after_discount > 300 else 40.0

    total = round(net_after_discount + delivery_charge, 2)
    return round(subtotal, 2), round(discount, 2), delivery_charge, total

def place_order():
    global next_order_id
    try:
        cid = read_positive_int("Enter customer ID: ")
        if cid not in customers:
            raise RecordNotFoundError(f"Customer ID {cid} not found.")
        rid = read_positive_int("Enter restaurant ID: ")
        if rid not in restaurants:
            raise RecordNotFoundError(f"Restaurant ID {rid} not found.")
        restaurant = restaurants[rid]
        if not restaurant["menu"]:
            print("[ERROR] This restaurant has no menu items yet.")
            return

        item_list = []
        while True:
            raw = input("Enter food item ID to add (0 to finish): ").strip()
            try:
                item_id = int(raw)
            except ValueError:
                print("[ERROR] Please enter a valid whole number.")
                continue
            if item_id == 0:
                break
            if item_id not in restaurant["menu"]:
                print("[ERROR] Invalid item ID for this restaurant.")
                continue
            qty = read_positive_int("Enter quantity: ")
            item_list.append((item_id, qty))

```

```

        if not item_list:
            raise InvalidInputError("Order must contain at least one item.")

except (InvalidInputError, RecordNotFoundError) as e:
    print(f"[ERROR] {e}")
    return

subtotal, discount, delivery_charge, total = calculate_bill(item_list, restaurant)
order_id = next_order_id
orders[order_id] = {
    "customer_id": cid,
    "restaurant_id": rid,
    "items": item_list,
    "status": ORDER_STATUSES[0],          # "Placed"
    "order_time": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    "subtotal": subtotal,
    "discount": discount,
    "delivery_charge": delivery_charge,
    "total": total,
}
next_order_id += 1
print(f"[OK] Order placed successfully. Order ID = {order_id} | Total = Rs.{total:.2f}")

def update_order_status():
    try:
        oid = read_positive_int("Enter order ID: ")
        if oid not in orders:
            raise RecordNotFoundError(f"Order ID {oid} not found.")
        print(f"Available statuses: {ORDER_STATUSES}")
        raw_status = read_non_empty("Enter new status: ")
        match = [s for s in ORDER_STATUSES if s.lower() == raw_status.lower()]
        if not match:
            raise InvalidInputError("Invalid status value.")
        new_status = match[0]
    except (InvalidInputError, RecordNotFoundError) as e:
        print(f"[ERROR] {e}")
        return

    orders[oid]["status"] = new_status
    print(f"[OK] Order {oid} status updated to '{new_status}'.")

def view_order_details(order_id=None):
    try:
        oid = order_id if order_id is not None else read_positive_int("Enter order ID: ")
        if oid not in orders:
            raise RecordNotFoundError(f"Order ID {oid} not found.")
    except (InvalidInputError, RecordNotFoundError) as e:
        print(f"[ERROR] {e}")
        return

    o = orders[oid]
    cust = customers.get(o["customer_id"], {"name": "Unknown"})
    rest = restaurants.get(o["restaurant_id"], {"name": "Unknown", "menu": {}})

    print(f"\n----- Order #{oid} -----")
    print(f"Customer      : {cust['name']}")

```

```

print(f"Restaurant      : {rest['name']}")
print(f"Order Time      : {o['order_time']}")
print(f"Status          : {o['status']}")
print("Items:")
for item_id, qty in o["items"]:
    item_name = rest["menu"].get(item_id, ("Unknown", 0, ""))[0]
    price = rest["menu"].get(item_id, ("Unknown", 0, ""))[1]
    print(f"    - {item_name} x{qty} @ Rs.{price:.2f}")
print(f"Subtotal          : Rs.{o['subtotal']:.2f}")
print(f"Discount           : Rs.{o['discount']:.2f}")
print(f"Delivery Chg       : Rs.{o['delivery_charge']:.2f}")
print(f"TOTAL              : Rs.{o['total']:.2f}")

# -----
# CUSTOM OPERATION 1 : COMPARE TWO ORDERS BY TOTAL AMOUNT
# -----
def compare_orders():
    try:
        oid1 = read_positive_int("Enter first order ID: ")
        oid2 = read_positive_int("Enter second order ID: ")
        if oid1 not in orders or oid2 not in orders:
            raise RecordNotFoundError("One or both order IDs do not exist.")
    except (InvalidInputError, RecordNotFoundError) as e:
        print(f"[ERROR] {e}")
        return

    t1, t2 = orders[oid1]["total"], orders[oid2]["total"]
    print(f"\nOrder #{oid1} total = Rs.{t1:.2f}")
    print(f"Order #{oid2} total = Rs.{t2:.2f}")
    if t1 > t2:
        print(f"Result: Order #{oid1} has the HIGHER bill (difference = Rs.{t1 - t2:.2f}")
    elif t2 > t1:
        print(f"Result: Order #{oid2} has the HIGHER bill (difference = Rs.{t2 - t1:.2f}")
    else:
        print("Result: Both orders have the SAME total amount.")

# -----
# CUSTOM OPERATION 2 : ORDER & RESTAURANT PERFORMANCE REPORT
# -----
def generate_report():
    if not orders:
        print("No orders placed yet. Report cannot be generated.")
        return

    total_orders = len(orders)
    all_totals = [o["total"] for o in orders.values()]
    total_revenue = sum(all_totals)
    avg_order_value = statistics.mean(all_totals)

    status_count = {}
    for o in orders.values():
        status_count[o["status"]] = status_count.get(o["status"], 0) + 1

    restaurant_revenue = {}
    for o in orders.values():
        restaurant_revenue[o["restaurant_id"]] = restaurant_revenue.get(o["restaurant_id"],
0) + o["total"]

```

```

best_restaurant_id = max(restaurant_revenue, key=restaurant_revenue.get)
best_restaurant_name = restaurants.get(best_restaurant_id, {}).get("name", "Unknown")

print("\n" + "=" * 55)
print("      ONLINE FOOD ORDER - PERFORMANCE REPORT")
print("=" * 55)
print(f"Report generated on : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f"Total customers      : {len(customers)}")
print(f"Total restaurants     : {len(restaurants)}")
print(f"Total orders placed    : {total_orders}")
print(f"Total revenue          : Rs.{total_revenue:.2f}")
print(f"Average order value   : Rs.{avg_order_value:.2f}")
print(f"Top performing rest.  : {best_restaurant_name}
(Rs.{restaurant_revenue[best_restaurant_id]:.2f})")
print("-" * 55)
print("Order status breakdown:")
for status in ORDER_STATUSES:
    print(f"    {status:<20}: {status_count.get(status, 0)}")
print("=" * 55)

# -----
# FILE HANDLING (PERSISTENCE)
# -----
def save_data():
    try:
        data = {
            "customers": customers,
            "restaurants": restaurants,
            "orders": orders,
            "next_ids": {
                "customer": next_customer_id,
                "restaurant": next_restaurant_id,
                "item": next_item_id,
                "order": next_order_id,
            },
        }
        with open(DATA_FILE, "w") as f:
            json.dump(data, f, indent=2)
        print(f"[OK] Data saved to '{DATA_FILE}'.")
    except OSError as e:
        print(f"[ERROR] Could not save data: {e}")

def load_data():
    global customers, restaurants, orders
    global next_customer_id, next_restaurant_id, next_item_id, next_order_id

    if not os.path.exists(DATA_FILE):
        print("[INFO] No saved data file found. Starting fresh.")
        return
    try:
        with open(DATA_FILE, "r") as f:
            data = json.load(f)
            customers = {int(k): v for k, v in data.get("customers", {}).items()}
            restaurants_raw = data.get("restaurants", {})
            restaurants.clear()
            for k, v in restaurants_raw.items():

```

```

        v["menu"] = {int(mk): tuple(mv) for mk, mv in v.get("menu", {}).items()}
        restaurants[int(k)] = v
    orders_raw = data.get("orders", {})
    orders.clear()
    for k, v in orders_raw.items():
        v["items"] = [tuple(pair) for pair in v["items"]]
        orders[int(k)] = v

    ids = data.get("next_ids", {})
    next_customer_id = ids.get("customer", 1)
    next_restaurant_id = ids.get("restaurant", 1)
    next_item_id = ids.get("item", 1)
    next_order_id = ids.get("order", 1)
    print(f"[OK] Data loaded from '{DATA_FILE}'.")
except (OSError, json.JSONDecodeError) as e:
    print(f"[ERROR] Could not load data: {e}")

```

```

# -----
# MAIN MENU
# -----
MENU = """
===== ONLINE FOOD ORDER MANAGEMENT & ANALYSIS SYSTEM =====
1. Register Customer
2. Add Restaurant
3. Add Food Item to Restaurant
4. View All Restaurants & Menus
5. Place Order
6. Update Order Status
7. View Order Details
8. Compare Two Orders (by Total Amount)
9. Generate Order & Restaurant Performance Report
10. View All Customers
11. Save Data to File
12. Load Data from File
13. Exit
=====
"""

```

```

def main():
    load_data()
    while True:
        print(MENU)
        choice = input("Enter your choice (1-13): ").strip()
        try:
            if choice == "1":
                register_customer()
            elif choice == "2":
                add_restaurant()
            elif choice == "3":
                add_food_item()
            elif choice == "4":
                view_restaurants()
            elif choice == "5":
                place_order()
            elif choice == "6":
                update_order_status()
            elif choice == "7":

```

```

        view_order_details()
    elif choice == "8":
        compare_orders()
    elif choice == "9":
        generate_report()
    elif choice == "10":
        view_customers()
    elif choice == "11":
        save_data()
    elif choice == "12":
        load_data()
    elif choice == "13":
        save_data()
        print("Thank you for using the Online Food Order Management System!")
        break
    else:
        print("[ERROR] Invalid choice. Please select 1-13.")
except Exception as e:
    # Final safety net so the menu never crashes on unexpected errors
    print(f"[UNEXPECTED ERROR] {e}")

if __name__ == "__main__":
    main()

```

Output:

Main Menu:

```
Command Prompt - python food_order_system.py

[INFO] No saved data file found. Starting fresh.

===== ONLINE FOOD ORDER MANAGEMENT & ANALYSIS SYSTEM =====
1. Register Customer
2. Add Restaurant
3. Add Food Item to Restaurant
4. View All Restaurants & Menus
5. Place Order
6. Update Order Status
7. View Order Details
8. Compare Two Orders (by Total Amount)
9. Generate Order & Restaurant Performance Report
10. View All Customers
11. Save Data to File
12. Load Data from File
13. Exit
=====
```

Registering Customers, Restaurants & Menu Items:

```
Command Prompt - python food_order_system.py

>> Registering customers, restaurants and menu items ...

Enter your choice (1-13): Enter customer name: Enter phone number: Enter email: Enter delivery address: [OK] Customer registered successfully. Customer ID = 1

Enter your choice (1-13): Enter customer name: Enter phone number: Enter email: Enter delivery address: [OK] Customer registered successfully. Customer ID = 2

Enter your choice (1-13): Enter restaurant name: Enter cuisine type: Enter rating (1.0 - 5.0): [OK] Restaurant added successfully. Restaurant ID = 1

Enter your choice (1-13): Enter restaurant name: Enter cuisine type: Enter rating (1.0 - 5.0): [OK] Restaurant added successfully. Restaurant ID = 2

Enter your choice (1-13): Enter restaurant ID: Enter food item name: Enter price (Rs.): Enter category ('Main Course', 'Starter', 'Beverage', 'Dessert'): [OK] Food item added. Item ID = 1

Enter your choice (1-13): Enter restaurant ID: Enter food item name: Enter price (Rs.): Enter category ('Main Course', 'Starter', 'Beverage', 'Dessert'): [OK] Food item added. Item ID = 2

Enter your choice (1-13): Enter restaurant ID: Enter food item name: Enter price (Rs.): Enter category ('Main Course', 'Starter', 'Beverage', 'Dessert'): [OK] Food item added. Item ID = 3

Enter your choice (1-13): Enter restaurant ID: Enter food item name: Enter price (Rs.): Enter category ('Main Course', 'Starter', 'Beverage', 'Dessert'): [OK] Food item added. Item ID = 4

Enter your choice (1-13): Enter restaurant ID: Enter food item name: Enter price (Rs.): Enter category ('Main Course', 'Starter', 'Beverage', 'Dessert'): [OK] Food item added. Item ID = 5
```

Viewing All Restaurants & Menus:

```
Command Prompt - python food_order_system.py

Enter your choice (1-13):
Restaurant ID: 1 | Spice Villa | Cuisine: South Indian | Rating: 4.5
ItemIDName      Price      Category
1   Masala Dosa   Rs.90.00  Main Course
2   Filter Coffee Rs.40.00  Beverage
3   Gulab Jamun   Rs.60.00  Dessert

Restaurant ID: 2 | Pizza Junction | Cuisine: Italian | Rating: 4.2
ItemIDName      Price      Category
4   Margherita Pizza Rs.320.00 Main Course
5   Garlic Bread    Rs.150.00 Starter
```

Placing Orders & Viewing Order Details:

```
Command Prompt - python food_order_system.py

Enter your choice (1-13): Enter customer ID: Enter restaurant ID: Enter food item ID to add (0 to finish): Enter quantity: Enter food item ID to add (0 to finish): [OK] Order placed successfully. Order ID = 1 | Total = Rs.190.00

Enter your choice (1-13): Enter customer ID: Enter restaurant ID: Enter food item ID to add (0 to finish): Enter quantity: Enter food item ID to add (0 to finish): [OK] Order placed successfully. Order ID = 2 | Total = Rs.576.00

Enter your choice (1-13): Enter order ID:
----- Order #1 -----
Customer      : Ananya Rajan
Restaurant    : Spice Villa
Order Time    : 2026-09-05 15:11:39
Status        : Placed
Items:
- Masala Dosa x1 @ Rs.90.00
- Gulab Jamun x1 @ Rs.60.00
Subtotal      : Rs.150.00
Discount      : Rs.0.00
Delivery Chg  : Rs.40.00
TOTAL         : Rs.190.00

Enter your choice (1-13): Enter order ID:
----- Order #2 -----
Customer      : Karthik Subramaniam
Restaurant    : Pizza Junction
Order Time    : 2026-09-05 15:11:39
Status        : Placed
Items:
- Margherita Pizza x2 @ Rs.320.00
Subtotal      : Rs.640.00
Discount      : Rs.64.00
Delivery Chg  : Rs.0.00
TOTAL         : Rs.576.00
```

Updating Order Status & Comparing Two Orders:

```
Command Prompt - python food_order_system.py

Enter your choice (1-13): Enter order ID: Available statuses: ('Placed', 'Preparing', 'Out for Delivery', 'Delivered', 'Cancelled')
Enter new status: [OK] Order 1 status updated to 'Preparing'.

Enter your choice (1-13): Enter order ID: Available statuses: ('Placed', 'Preparing', 'Out for Delivery', 'Delivered', 'Cancelled')
Enter new status: [OK] Order 2 status updated to 'Out for Delivery'.

Enter your choice (1-13): Enter first order ID: Enter second order ID:
Order #1 total = Rs.190.00
Order #2 total = Rs.576.00
Result: Order #2 has the HIGHER bill (difference = Rs.386.00)
```

Performance Report, Customer List & Save:

```
Command Prompt - python food_order_system.py

Enter your choice (1-13):
=====
      ONLINE FOOD ORDER - PERFORMANCE REPORT
=====
Report generated on : 2026-09-05 15:11:39
Total customers      : 2
Total restaurants    : 2
Total orders placed  : 2
Total revenue        : Rs.766.00
Average order value  : Rs.383.00
Top performing rest. : Pizza Junction (Rs.576.00)
-----
Order status breakdown:
Placed              : 0
Preparing           : 1
Out for Delivery    : 1
Delivered           : 0
Cancelled           : 0
=====

Enter your choice (1-13):
ID   Name                Phone      Email                Address
-----
1    Ananya Rajan          9876543210  ananya.r@example.com 12 Lake View Street, Chennai
2    Karthik Subramaniam    9123456780  karthik.s@example.com 45 MG Road, Bengaluru

Enter your choice (1-13): [OK] Data saved to 'food_order_data.json'.

Enter your choice (1-13): [OK] Data saved to 'food_order_data.json'.
Thank you for using the Online Food Order Management System!
```